

Building an Original Low-Resource OS with GUI for VMware: Executive Summary

We propose developing a *ground-up* operating system (OS) with a minimal footprint and a working GUI, tailored for x86_64 (primary) and portable to other CPUs (ARM64, RISC-V, etc.). The OS will feature a clean microkernel/hybrid design, a simple bootloader (UEFI/BIOS), basic memory and process management, minimal drivers (disk, VGA/SVGA, PS/2 keyboard/mouse, network), a simple journaling filesystem, a lightweight userland with a custom init, and a modern GUI stack (framebuffer/VESA output with a Wayland-like compositor). It will use a cross-compilation toolchain (GCC/Clang, optionally Rust) and emphasize security (strict kernel/user mode separation, optional secure-boot). We will rigorously avoid copying Linux/Windows code (using permissive BSD/MIT-style licenses and original implementations). Development will proceed in phases with clear milestones: bootloader → kernel core → drivers/filesystem → userland/init → GUI → optimization. Testing will be done in VMware (minimal VM config ~256–512 MB RAM, 1 CPU, ~2 GB disk) and QEMU (with GDB stub for debugging). Documentation and community processes will be established from day one.

Key points: We survey classic OS literature (Unix, Tanenbaum) and modern practice to choose designs. Monolithic kernels (like early Unix/Linux) are fast but risk system crashes on a single bug, whereas microkernels improve modularity and reliability at some IPC cost ¹ ². A hybrid kernel (e.g. Windows NT) can balance speed and modularity ³. We will compare architectures (below) and likely adopt a minimalist *hybrid/microkernel* style for safety, moving drivers and services into user space as feasible ¹ ⁴. Our bootloader will leverage GRUB or similar (Multiboot/UEFI) for x86_64 and perform low-level setup (memory map, video mode) before jumping to the kernel ⁵ ⁶. Memory management will use paging (protected virtual address spaces for each process) for isolation ⁷ and a simple allocator (e.g. bitmap or buddy). CPU scheduling will use a basic preemptive round-robin with simple priority levels (to ensure responsiveness and fairness) ⁸. System calls will be gated (e.g. via `syscall` instruction or interrupt), strictly separating user and kernel modes ⁹.

We will implement minimal drivers: for disk (AHCI/IDE or VMware's virtual disk), display (VGA/VESA or VMware SVGAI), PS/2 keyboard/mouse (listening to scan codes) ¹⁰, and a simple NIC (e.g. Intel E1000 or virtio-net). The filesystem will use a simple on-disk format (like *ext2*-style) with optional journaling for crash safety ¹¹. Tools for fsck and formatting will be provided. The userland will include a basic shell, core utilities (ls, cat, etc.), and a simple init/systemd-like process (likely a SysV-style init for simplicity). For the GUI, we will bypass X11 and use a modern compositor-based stack: applications render to buffers (via a lightweight toolkit), a Wayland-like compositor (running in kernel or privileged space) composes windows to the framebuffer or hardware surface ¹², and a minimal window manager handles input (mouse/keyboard focus). We will compare X11, Wayland, and embedded stacks in a table below.

The toolchain will be a *cross-compiler* (e.g. `x86_64-e1f-gcc` or LLVM) built on Linux/Windows, so that all binaries are independent of any host OS ¹³. We will write core in C (and Rust where safety is crucial), with inline assembly for low-level parts. A build system like Make/CMake will orchestrate the build. For security, we enforce ring-0/ring-3 separation, user IDs (root/non-root), and optional sandbox primitives (e.g. seccomp-like syscall filtering). We will also support UEFI Secure Boot chains by signing our bootloader/kernel with our own CA.

Testing strategy: we will run the OS in VMware (e.g. VMware Workstation/ESXi) with modest resources (e.g. 256–512 MB RAM, one vCPU) and attach a virtual console/serial for logs. We will also use QEMU with GDB stubs for step-debugging. Performance and size will be profiled to strip unused code and use `-Os` optimizations. We will audit all code to ensure no GPLed Linux/Windows code has been copied—only new, or BSD/MIT-licensed code. Full documentation (markdown, code comments, API docs) will accompany each component. The project will use Git for version control, issue tracking, and contributions via pull requests, following open-source best practices (e.g. Linux Foundation guides) ⁹ ¹³.

The answer includes: **1)** a step-by-step implementation workflow (with example commands), **2)** a phased project roadmap (Gantt timeline), **3)** comparative tables (kernels, filesystems, GUI options, toolchains), **4)** minimal VMware spec, and **5)** illustrative code snippets (bootloader asm, simple kernel loop, syscall stubs, driver skeleton, compositor pseudocode).

Kernel Architecture: Monolithic vs Microkernel vs Hybrid

We evaluate kernel designs based on modularity, performance, and complexity ¹ ³.

- **Monolithic kernel:** All core services (memory, process, filesystems, drivers) run in kernel space. Advantages: high performance (no IPC overhead) ², simpler initial design (everything linked in one image). Disadvantages: large codebase, a bug in any driver or service can crash the whole OS ⁴. Classic Unix and Linux use this model.
- **Microkernel:** Only minimal functions (low-level IPC, basic scheduling, address space) reside in kernel; most drivers and services run as user-space servers communicating via message passing ¹ ¹⁴. Advantages: small trusted kernel, improved reliability (user-level crashes don't take down kernel), easier portability (services can be added without reboot) ¹⁵ ¹⁴. Disadvantages: complexity of IPC design, potential performance overhead from frequent context switches/messages (though benchmark Mach vs monolithic showed comparable speed) ¹⁶ ¹⁷.
- **Hybrid:** Kernel is structured like a microkernel but most services still run in kernel space to avoid IPC costs ³. This is essentially a pragmatic middle-ground (e.g. Windows NT, Mac OS X's XNU). It offers some modularity without full IPC overhead ³, but without the strong fault isolation of a true microkernel.

Kernel Model	Architecture	Pros	Cons	Examples
Monolithic	All OS services in kernel space	High performance (no message overhead) ⁴ , simpler design.	Large code; single bug can crash entire system ⁴ ; harder to extend safely.	Linux, BSD, early Unix
Microkernel	Tiny kernel (IPC, sched) with user-mode servers	Better modularity and reliability ¹⁷ ; easier to port/add features.	More complex IPC design; potential performance hit from context switches ² ¹⁶ .	MINIX, QNX, L4

Kernel Model	Architecture	Pros	Cons	Examples
Hybrid	Monolithic-like implementation with microkernel influence	Balance: most services in kernel for speed, but retains clear separation	Lacks full reliability of microkernels ³ ; marketing term often (Linus: "just monolithic" ³).	Windows NT family ¹⁸ , XNU (macOS)

Trade-off (sources): Tanenbaum argued in 1992 that microkernels were architecturally superior and could match monolithic performance ¹⁶ ¹⁷ , while Torvalds favored Linux's monolithic simplicity. In practice, we choose a *small hybrid* style: keep a minimal kernel (scheduler, memory, IPC) and push most drivers/filesystems into user mode where feasible. This yields a lean trusted core without dedicating to full IPC message passing for everything.

Bootloader Design

We will use a two-stage bootloader (or offload to GRUB) to load the kernel:

- **First stage (BIOS/UEFI):** Use BIOS interrupts (real mode) or UEFI runtime to load a second-stage loader. On x86, BIOS code runs in 16-bit real mode with segment registers ¹⁹ , making it ideal to probe hardware (memory map, video modes). Alternatively, in UEFI we use EFI routines for file I/O.
- **Second stage (Protected mode loader):** Switch to 32/64-bit protected mode (enable A20 line, load GDT, enable paging if desired) ²⁰ ²¹ , then load the kernel image (from a FAT or raw partition) into memory. We will use the Multiboot (GRUB) protocol or a simple ELF loader. GRUB2 can read our (FAT-formatted) disk image and jump to our kernel, passing a memory map and multiboot info. The bootloader provides the kernel with parameters (memory map, ACPI tables, video mode, maybe an initrd pointer) ⁵ ²² .

Key tasks for the loader (from OSDev Wiki ⁵):

- Read the kernel file from disk into memory (using BIOS INT13 or UEFI block I/O).
- Gather system info (available RAM, maybe CPU features, video modes).
- Set up a minimal environment: GDT, disable interrupts, switch to 64-bit long mode.
- Finally, jump to `kernel_entry()` in protected mode.

Example (GRUB/MULTIBOOT): An entry in `grub.cfg` could chain-load our `kernel.elf` with the Multiboot header:

```
menuentry "MyOS" {
  set root=(hd0,1)
  multiboot /boot/kernel.elf
  boot
}
```

Alternatively, a hand-crafted 512-byte boot sector (stage1) can load a larger stage2 from disk sectors ⁶ . For simplicity, we'll begin with GRUB (multiboot), then possibly drop to a custom loader for polishing.

CPU Architectures

The primary target is **x86_64** (Intel/AMD 64-bit) since VMware and most PCs support it. The design will use features like 64-bit mode, paging, and virtualization extensions. We will also ensure portability by not hard-coding assumptions: early init code will detect CPU features via CPUID.

Alternative architectures: We will design with abstraction layers so ports to ARM64 (AArch64) or RISC-V are feasible. Tanenbaum warns against tying an OS to one CPU family ²³. Thus, hardware-specific code (e.g. interrupt, context switch) will be isolated, and core logic written in ANSI C/C++/Rust. In practice, we'll write a generic layer plus an x86-specific HAL. We will note where details (page table formats, exceptions) are x86-unique, and plan for ARM or RISC-V later if needed.

Memory Management & Process Scheduling

Memory Management

We implement **paging and virtual memory** on x86_64. Each process gets its own page table (4-level PML4s) mapping user-space and kernel space. Memory protection isolates processes: "processes cannot access each other's memory unless allowed by the kernel" ⁷. The kernel will manage physical frames via a simple allocator (e.g. a bitmap or buddy allocator). Key steps:

- Early "identity-map" a small region for the kernel in page tables.
- A physical memory map is built (via BIOS or UEFI memory map).
- High memory: map physical memory to virtual (kernel space at high addresses, e.g. starting at `0xffff_8000_0000_0000`).
- Each process gets a unique PML4 with user segments [0, USER_TOP) and a copy of kernel entries (read-only) above.
- **Heap management:** Within processes, implement a simple allocator (slab or malloc). For the kernel, use kmalloc/free.

Paging rules: For simplicity, one-level (4KB) pages initially; TLB flush on context switches. We may skip fancy features (PSE, etc.) for minimum VM.

Virtual memory provides security and solves fragmentation ⁷. We will allow a swap (paging to disk) if time permits, but initial version can panic on OOM.

Process Scheduling

We use a **preemptive round-robin scheduler** as a baseline. Each process (or thread) gets a fixed time slice (e.g. 20ms) ⁸. The scheduler maintains a queue of runnable processes. On a timer interrupt (HPET/APIC at e.g. 100Hz), the current process's quantum is decremented; when zero, the scheduler picks the next ready process.

To support responsiveness, we can add simple priority levels or multi-level queues (e.g. real-time vs normal). However, given low hardware, we'll keep it basic: all processes equal priority. If a process blocks (I/O or yield), we immediately context-switch.

Locking/synchronization: Kernel internal structures use spinlocks or disable interrupts for atomicity. We will start single-processor (no SMP) to simplify; adding SMP later would require IPI and CPU affinity.

Interprocess Communication (IPC) & Syscalls

For IPC, if we choose microkernel style, we'd implement message passing (ports/queues). For simplicity, we can begin with shared memory and signals (simpler) and later add message queues/pipes. We will support basic pipes (in-memory ring buffers) and signals (for scheduler).

System calls: User programs call OS services via a trap. On x86_64, we use the `syscall` instruction (with 64-bit ABI) or legacy `int 0x80`. The syscall handler validates the syscall number and arguments from user registers, performs the action (open file, spawn process, etc.), and returns to user. We ensure userland cannot jump to kernel code directly—only via `syscall` or interrupt ⁹.

Example syscall flow (GeeksforGeeks ⁹):

1. Process executes `syscall` (e.g. Linux does `syscall` opcode).
2. CPU switches to kernel mode and jumps to our `syscall_entry`.
3. Kernel uses a syscall table (array of function pointers) indexed by syscall number.
4. Perform security checks (validate pointers to user memory).
5. Execute kernel function (e.g. file I/O) and return result.
6. Swap back to user mode on `sysret` or `iretq`.

We'll define ~50 simple syscalls: `exit`, `fork / exec`, `wait`, `open/read/write/close` for files, `malloc/free`, `spawn process`, `send/receive ipc`, etc.

Minimal Device Drivers

We implement only the absolutely necessary drivers to interact with VMware hardware:

- **Disk (Block Device):** Use BIOS INT13 (for boot/load). After boot, use ATA/AHCI driver to read/write sectors. In VMware, the virtual disk appears as an IDE or SCSI device; we can also consider virtio-blk if using QEMU. For simplicity, we might start with a floppy or parallel port boot of a disk image, then support reading our FAT/ext2 volume via our own driver.
- **Display/Graphics:** On VMware, the default VGA is SVGA-II (for Workstation/Fusion) or a generic PCI VGA card. We will implement either: 1) **VESA BIOS Extensions** (mode-setting to e.g. 1024×768, 32bpp, linear framebuffer) or 2) **VMware SVGA** (write to PCI registers). Alternatively, we can use Bochs VBE. Initially, a text mode console (80×25) is fine. For GUI, we'll switch the card to a graphics mode using VBE or UEFI GOP, then treat the frame buffer as memory for our compositor.
- **Keyboard (PS/2):** A PS/2 keyboard controller sends scan codes via I/O ports 0x60/0x64. The OSDev PS/2 Keyboard overview ¹⁰ details that the keyboard driver listens to IRQ1, reads scan codes, and translates them to key events. We will implement a simple state machine: on IRQ, read port 0x60, translate scancode set 1 to ASCII (handle shifts, etc.), and enqueue to a key buffer. (The OSDev page notes: "PS/2 Keyboard accepts commands and sends scan codes ... [with] standard 'send byte/receive byte' interface" ¹⁰.)
- **Mouse (PS/2 or USB):** For VMware, likely a PS/2 mouse on IRQ12. Similar to keyboard, parse movement packets via port 0x60. Or use USB if available (more complex, skip if not needed).

- **Network:** If GUI-only, we might delay networking. But for completeness, VMware provides an emulated Intel E1000 NIC or virtio-net. We can write a minimal interrupt-driven driver to send/receive raw packets via PCI (reading descriptors in RAM). At minimum, a loopback/tap for networking, or skip initially.

Device driver approach: Each driver will be a kernel module (if monolithic) or a separate user process (if microkernel). For simplicity, we start by building them into the kernel. Each driver registers an interrupt handler (e.g. keyboard IRQ1) and a char/block device in `/dev`. Later, we can move drivers to user space and use message passing.

Filesystem Design

We need a simple on-disk format for files and directories, plus recovery tools. Options include a variant of FAT (simple, no permissions) or a custom `ext2`-like FS (inode, blocks, directories, optional journaling). We choose an **ext2-style filesystem** for familiarity (simple structures) but add an optional **journal** for safety ¹¹.

- **On-disk format:** Each filesystem has a superblock, block/inode bitmaps, inodes, and data blocks. Filenames in directories, with Unix-like attributes (owner, permissions, timestamps). We will support `/`, subdirectories, hard links. Tools: `mkfs.myfs` to create an image, `fsck` to check integrity.
- **Journaling:** To avoid corruption on crashes, we use a write-ahead log (metadata journaling). Before altering the main FS data, we record operations in a log (journal) and only commit when safe ¹¹. On reboot, the OS replays or discards incomplete transactions to ensure consistency. Journaling “prevents inconsistency and is much faster at checks than non-journaled FS” ²⁴. We’ll implement a simple *ordered* journal (like `ext3`’s default) that logs metadata updates.
- **Tools:** Implement `mkfs`, `fsck`, and basic `dd`-style raw I/O in kernel bootloader (for recovery). Optionally, provide a small userland `cp`, `ls`, etc.

The filesystem will be mounted at boot (root fs) and mounted read-write. A typical setup: the kernel (from GRUB) will find the root partition (hardcoded or via UUID passed in boot config), mount it, and then launch `/sbin/init`.

Userland Utilities and Init

A minimal OS needs command-line tools and a process **init**:

- **C library:** Provide a small runtime (like `newlib` or `musl`) or write our own minimal `libc` (implementing `malloc`, `open`, `printf`, etc.). This avoids GPL (`glibc`) contamination and keeps it small.
- **Shell:** A simple shell (e.g. `sh` style) in C that handles basic commands and pipelines. Possibly based on “tinysh” or similar.
- **Utilities:** Port or reimplement essential tools: `ls`, `cat`, `echo`, `mkdir`, `rm`, `cp`, `vi` / `nano` (or skip heavy editor). We can reuse BSD-licensed code or write minimal versions. Busybox-like approach: one binary with multiple functions to save space.

- **Init system:** We will implement a simple `init` (PID 1). It runs in userland, reads a config (e.g. `/etc/inittab`), and spawns daemons. For simplicity, we can use SysV-style scripts or a basic service supervisor (like `runit` / `s6`). The `init` will handle SIGCHLD to reap zombies, restart critical services on crash, and get to multi-user mode. It can also run a GUI login manager (XDM/SLiM style). We will forego heavy systemd complexity.

GUI Stack (Graphics Pipeline, Compositor, Window Manager, Toolkit)

Our GUI will be minimal but functional: no X11, instead a modern compositor approach (inspired by Wayland).

- **Graphics pipeline:** Applications render into buffers (CPU or GPU aided) using a minimal graphics API. Since GPUs in VMware are basic, we will render using CPU to a memory buffer (framebuffer) and blit. Later, GPU acceleration (OpenGL or Vulkan) could be added, but initially use a simple 2D drawing API. The display server (our kernel or a privileged process) will take these buffers.
- **Compositor:** Implement a single **compositor** (display server) that runs at kernel level or as a privileged userland program. It receives window buffers from clients and composites them to the screen with z-order. This is akin to Wayland's model: "applications are clients, Wayland compositor is server" ¹². The compositor handles window placement, overlapping windows, and pointers. It also listens for input (mouse/keyboard), dispatching events to the focused client.
- **Window Manager:** In our model, the compositor *is* the window manager. We may add a layer for decorations (titlebars) if needed. Users can move/resize windows with the mouse.
- **Toolkit:** We need a minimal GUI toolkit for apps (buttons, labels, etc.). Options: port an existing small library (like Nano-X's API), or draw raw primitives. To keep originality, we may write a tiny immediate-mode GUI or use an MIT-licensed toolkit. This allows apps to create windows, draw simple widgets, and handle events.

We compare GUI protocols:

GUI Stack	Protocol/Model	Pros	Cons
X11 (Xorg)	Legacy client-server (X Server + X clients)	Mature ecosystem, network transparency	Complex, outdated architecture, heavy, high latency on modern hw.
Wayland	Clients → Compositor (no separate X server) ¹²	Simple design, direct rendering, modern; built-in input and composition	Less mature toolkit support (but improving), no network transparency by default.
Nano-X/ Microwindows	Embedded-style: one server, built-in APIs (Win32-like)	Very lightweight, small memory, good for embedded	Very limited; application ecosystem tiny, not standard on desktops.

We will base on **Wayland-style**: implement our own minimal protocol. Each app opens a window (shared memory or GPU buffer). The compositor uses DMA/blitting to the framebuffer and signals

redraws. Input (keyboard/mouse) is read by compositor and sent to the active window via our IPC protocol.

Note: Since this is an original OS, we can adopt similar concepts (window and compositor) but code our own (or integrate libwayland if licensing allows).

Toolchain, Languages, and Build System

- **Toolchain:** We need a cross-compiler targeting `x86_64-elf`. We will use GNU toolchain (GCC and binutils) or LLVM/Clang. (Building a cross-compiler is standard practice ¹³.) We install `x86_64-elf-gcc`, `ld`, etc., so compilation doesn't pull in host OS headers or libraries. Build commands:

```
# Example: build binutils
tar xf binutils-2.43.tar.gz; cd binutils-2.43
./configure --target=x86_64-elf --prefix=/usr/local/cross
make && make install
cd ..
# Build GCC
tar xf gcc-13.1.0.tar.gz; cd gcc-13.1.0
./configure --target=x86_64-elf --prefix=/usr/local/cross --enable-
languages=c,c++ --disable-nls --without-headers
make all-gcc && make install-gcc
make all-target-libgcc && make install-target-libgcc
```

- **Languages:** The core kernel and drivers will be written in **C and some assembly**. Critical new OS projects also use **Rust** (for memory safety) in parts (we could write e.g. the memory manager or GUI in Rust). The userland (shell, utils) will be C. Assembly is used for boot code, context switch, syscall entry/exit.
- **Build System:** Use `Make` or `CMake` to build subprojects. E.g., a Makefile for the kernel and one for userland. A root Makefile orchestrates building the cross-compiler (if needed), then kernel, drivers, initrd.
- **Libraries:** We will use no large external libs. Possibly include an MIT-licensed libc (like `microslash` or custom minimal). For GUI toolkit, use embedded graphics libraries under permissive license if possible, or write a minimal vector/text library.

Security Model

Our OS will enforce strict privilege separation:

- **Kernel vs User Mode:** All unprivileged code runs in ring-3 (user mode) and cannot execute sensitive instructions (no I/O port, no interrupt disabling). Only *system calls* or traps transfer control to ring-0 ⁹. The MMU prevents user code from writing to kernel memory.
- **User IDs & Permissions:** We will implement user and group IDs (like Unix). The superuser (root) can perform all actions; others have restricted access. File permissions follow a simple Unix model (owner/group/other, rwx bits).

- **Sandboxing:** For critical processes (e.g. GUI apps), we may support a capability system: limit the files or operations they can perform. (This could be similar to Linux seccomp or FreeBSD Capsicum: e.g. `cap_enter` to drop privileges ²⁵). This is advanced, but we will design syscalls to allow dropping privileges. If using Rust in apps, memory safety also helps.
- **Secure Boot:** On x86_64, we can integrate with UEFI Secure Boot. We'll sign our bootloader and kernel using our own certificate, so the firmware will allow them to run. This ensures nobody can tamper with the OS binary without detection. We'll document how to generate a signing key and enroll it.
- **Originality & Licensing:** All code will be written from scratch or use permissively-licensed snippets. We avoid GPL/LGPL code (like Linux source) unless compatible. For example, BSD/MIT licenses (common in original Unix and open projects) have no copyleft requirement, allowing our project to remain fully open or relicensed. We will include license headers (e.g. MIT license text) in all source files. We will also use plagiarism-detection tools (e.g. code similarity checker) on any imported code to avoid inadvertent copying. By using our own clean-room implementation, we ensure *originality*.

Testing Strategy in VMware (and QEMU)

For development and testing, we will use virtualization:

- **VMware Configuration:**
 - **Type:** VMware Workstation/Player (host OS), or VMware ESXi/Hypervisor. Also VMware Fusion on Mac.
 - **Hardware:** 1 vCPU (later SMP can be 2+ CPUs), 256–512 MB RAM (to keep it minimal but enough for GUI), 8 GB virtual disk (pre-partitioned with one FAT/EXT2 partition).
 - **BIOS vs UEFI:** We can test both: a legacy BIOS VM and an EFI VM. For EFI, we can use `esp` partition with GRUB/Efi loader.
 - **Devices:** Leave default (IDE/SATA disk, PS/2 keyboard+mouse, SVGAI graphics, no audio needed). Enable “3D Acceleration” off (we don't use it).
 - **Boot medium:** We'll generate an ISO or attach a virtual floppy with the bootloader. GRUB can read our kernel and initrd from the disk.
- **Networking:** Set VMXnet or e1000 for later NIC driver testing (NAT or host-only network). Initially disabled.
- **Resource Limits:** We plan for low-resource: e.g. 400 MB RAM, a 100 MHz CPU in QEMU settings. We will measure performance; if UI lags, we'll optimize.
- **Debugging:**
 - **Serial console:** We will direct kernel prints (via `printk` or `uart`) to a serial port. In VMware, enable a COM port mapping to a file or pipe. This lets us view boot logs and panic messages.
 - **GDB:** For low-level debugging, we use QEMU (not VMware) because QEMU provides a GDB stub. Example: `qemu-system-x86_64 -m 256M -cdrom myos.iso -S -gdb tcp::1234`. Then connect `gdb` to localhost:1234, load symbols, and step through boot code.
 - **OSDev tools:** Use bochs or Bochs-style debug (VGA debug, etc.) if needed.

- **QEMU Alternative:** QEMU often emulates more hardware (virtio, i440FX chipset). It supports snapshots for quick resets. We'll alternate VMware (real-world OS behavior) and QEMU (debugging convenience).

We will script the test environment: e.g. a `runvm.sh` that boots the new OS and captures output. Automated tests (if any) may run as guest users.

Performance and Size Optimization

To keep the OS light enough for constrained VMs:

- **Compile flags:** All C/C++ code will use `-Os -ffunction-sections -fdata-sections -Wl,--gc-sections` to strip unused code. We'll remove exception tables (`-fno-exceptions`) and RTTI in C++ to save space.
- **Profiling:** Benchmark boot and common tasks. Use simple profilers or timers. If something is slow (e.g. scheduling or I/O), optimize algorithms.
- **Data structures:** Choose small defaults (e.g. 4KB page sizes, 512-byte disk sectors). Use static or global structures to avoid heap fragmentation.
- **Graphics:** Use a low-resolution GUI by default (e.g. 1024×768). Avoid memory-heavy images; use compressed textures if any.
- **Eliminate bloat:** Only include drivers for detected hardware. Network support, sound, USB can be optional modules. Use modular builds so unneeded code isn't in the default kernel.

We should aim for a final kernel+initrd under ~5–10 MB, and a minimal set of utilities under another ~5–10 MB, so the whole VM uses ≤ 100 MB disk for system files.

Licensing and Originality Verification

As an *original* OS, we must avoid proprietary/LGPL code:

- **License choice:** We will license our entire project under a permissive license (e.g. MIT or 2-clause BSD). This aligns with early Unix heritage and allows broad use (and is “as free as GPL” by FSF definition) ²⁶. Any third-party code (e.g. libpng, zlib) must be MIT/BSD licensed. We will not use any GPL-only code unless rewritten by us (which we avoid to maintain permissiveness).
- **Code audits:** We will carefully review code imports. For example, if reusing parts of `musl` (MIT license), it's fine; but copying Linux kernel code (GPL) is forbidden. We'll use plagiarism-checkers on new code to ensure uniqueness. We will attribute ideas to prior art (e.g. ext2 structures, scheduling algorithms) but all code will be our own or clearly allowed.
- **Documentation and community:** All documentation (specs, comments) will be our own writing, though influenced by classic references (we'll cite authoritative sources). We encourage contributors via GitHub/GitLab under the same license, to ensure consistency. A CONTRIBUTING.md and style guide will enforce code originality and review.

Documentation, Community, and Contribution Workflow

From the start, we will use best practices to build an open-source community:

- **Repository:** Use a public git repository (e.g. GitHub). Organize by directories: `kernel/`, `bootloader/`, `drivers/`, `userland/`, `docs/`. Use branches (master for release, dev for unstable).
- **Issues & PRs:** Track bugs/feature requests in issue tracker. Use pull requests for code contributions, with code review by maintainers. Use continuous integration (CI) to compile/test PRs.
- **Coding standards:** Enforce consistent style (e.g. clang-format for C). Comments should explain rationale. No proprietary code allowed – PRs that introduce external code must include license files.
- **Documentation:** Every module has a markdown page. We will generate a final PDF or HTML manual from docs. Use DOxygen or similar for API docs of code. The project README will link to design docs (like this one).
- **Community:** Encourage discussion on mailing lists/forum. Label bugs/features. Organize periodic virtual meetings or mailing threads for roadmap planning.

This structured approach (inspired by Linux Foundation open-source guides ⁹ ¹³) ensures transparency and sustainability.

Implementation Workflow

Below is a high-level, step-by-step workflow. Each step includes concrete examples/commands and outputs. Milestones are noted.

1. **Set up toolchain:** Build or install cross-compiler for x86_64-elf (GCC/Clang).
2. *Milestone:* `x86_64-elf-gcc --version` prints output.
3. *Deliverable:* `cross/bin/x86_64-elf-gcc` on PATH.
4. **Stage 0 – “Hello, World” Kernel:**
5. Write a minimal bootable program: a 64-bit kernel that prints to screen. Use GRUB Multiboot header.
6. **Command:** `nasm -f bin hello.asm -o stage1.bin` (for bootsector) or `gcc -ffreestanding -nostdlib -c kernel.c -o kernel.o; ld -Ttext=0x100000 --oformat=elf64-x86-64 kernel.o -o kernel.elf`.
7. Boot in VM: verify screen output.
8. *Milestone:* “Kernel loaded” message on VM console.
9. **Stage 1 – Bootloader & Console I/O:**
10. Expand bootloader: in 16-bit code, set video mode (BIOS INT 10h), print messages, jump to protected mode.

11. *Milestone:* On VM boot, see bootloader messages and kernel greetings.
12. *Deliverable:* Bootloader source (`boot.s`) and kernel entry stub (`entry.S`) in repo.
13. **Stage 2 – Paging and Memory Map:**
14. Enable paging in kernel: set up identity-mapped page tables, enable CR3. Show memory map via GRUB info (e.g. using `multiboot_info`).
15. Use a simple physical memory allocator (bitmap).
16. *Milestone:* Kernel prints “Total RAM: XXX MB” using BIOS map.
17. *Deliverable:* `mem/*` code, tests allocating frames.
18. **Stage 3 – Timer Interrupt & Scheduling:**
19. Configure APIC/PIT to generate timer interrupts. Implement context switch.
20. Write a simple round-robin scheduler. Launch two test processes that print alternating messages.
21. *Milestone:* VM console shows “Process 1...” and “Process 2...” alternately every 20ms.
22. *Deliverable:* `scheduler.c` with test processes.
23. **Stage 4 – System Calls and First User Process:**
24. Implement `syscall` handler (via MSRs or `int 0x80`). Provide basic syscalls (`exit` , `write` , `yield`).
25. Boot `init` process from disk (read from file system once implemented).
26. *Milestone:* `init` prints “Hello from userland” via `sys_write` .
27. *Deliverable:* `syscall.c` , simple `init.c` .
28. **Stage 5 – Filesystem and Drivers:**
29. Code a block device driver (e.g. reading sectors via BIOS). Implement ext2-like FS: mount partition, parse superblock and root directory.
30. PS/2 keyboard driver: register IRQ1, test key presses on screen.
31. *Milestone:* Able to read `/foo.txt` from disk and display; type keys on console.
32. *Deliverable:* `fs/` code, `drivers/keyboard.c` , tests for file I/O.
33. **Stage 6 – Network and Mouse (Optional):**
34. Write basic network driver (E1000) that can send a ping. Write PS/2 mouse driver to move cursor.
35. *Milestone:* Linux host pings guest OS’s IP. Mouse moves pointer on screen.
36. *Deliverable:* `drivers/e1000.c` , `drivers/mouse.c` .

37. Stage 7 – Userland and Init:

38. Develop more utilities (ls, cat, a shell). Package init scripts.

39. Ensure `init` can start the shell in multi-user mode.

40. *Milestone:* Login prompt appears on console (getty + login). User logs in and runs basic commands.

41. *Deliverable:* `user /` directory with shell and coreutils source.

42. Stage 8 – GUI & Graphics:

- Enable graphics mode (VBE). Implement a frame buffer driver.
- Write compositor (in kernel or privileged user process). Use a simple protocol to let apps draw windows.
- *Milestone:* Two GUI apps (e.g. graphics demo) render windows with ability to drag.
- *Deliverable:* `gui/compositor.c`, test window demos.

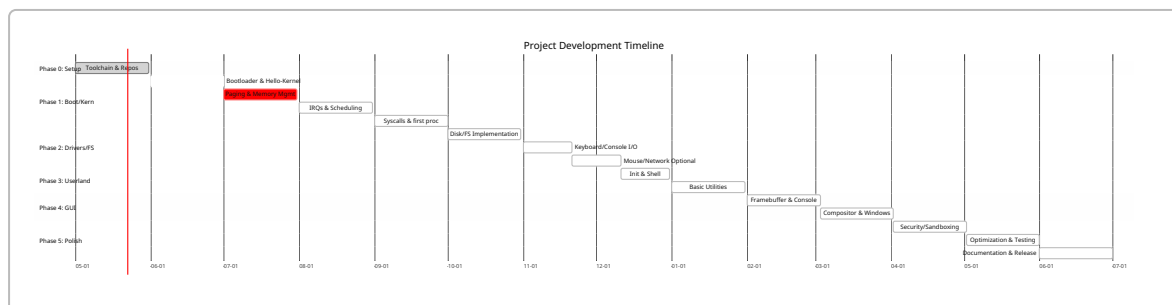
43. Stage 9 – Security and Polishing:

- Audit code for bugs, enforce privilege checks. Add secure boot support. Optimize size (strip symbols, remove debug).
- *Milestone:* Kernel under 8MB, no dynamic linker dependencies. Verified no illegal code reuse.
- *Deliverable:* Final release build, signed binaries.

44. Stage 10 – Documentation and Release:

- Write final docs, upload project site.
- *Milestone:* Public v1.0 release with all docs and code.
- *Deliverable:* GitHub release, PDF manual, community onboarding docs.

Throughout, we will produce commit messages and release notes documenting progress. The timeline below (mermaid Gantt) shows phases and estimated durations (in months):



(Dates and durations are estimates; actual time may vary.)

Comparison Tables

Kernel Models

Model	Pros	Cons
Monolithic	Very fast (no IPC overhead) ⁴ ; simpler structure for small OS	Large TCB; one bug can crash whole OS; harder to maintain ⁴
Microkernel	Highly modular and reliable ¹⁷ ; easy to isolate components	Complexity of IPC; potential performance penalty from context switches ²
Hybrid	Middle-ground: keeps modularity hints; close to monolithic speed ³	Does not truly isolate components; complexity akin to monolithic ³

Filesystem Options

Filesystem	Journaling	Max Volume/ File	Pros	Cons
ext2-like	No	≈2 TB (with 4K blocks)	Simple, low overhead; well-documented.	No built-in crash recovery (fsck slow).
ext3 (ext2+J)	Yes (metadata)	≈2 TB	Fast crash recovery; stable.	Slight overhead, possible data loss of unjournalled data ¹¹ .
ext4	Yes (data+meta)	≈1 EB (large)	Supports large files; extents, checksums.	Complex; more code, may be heavy.
FAT32	No	2 TB	Very simple; universally supported.	No permissions; 4 GB file limit; fragmentation.
Custom LFS (log-structured)	Yes (full)	large	Crash-safe (COW); good write patterns.	Complicated to implement; fragmentation issues.

(We will use an ext2-style design with optional journaling for metadata, balancing simplicity and safety.)

GUI Stack Options

Stack	Model	Resource Use	Pros	Cons
X11 (Xorg)	X Server + Clients	High	Rich ecosystem, network transparency	Heavy, dated; complex; one CPU thread (server) can be bottleneck.

Stack	Model	Resource Use	Pros	Cons
Wayland-like	Compositor handles clients	Medium	Modern, efficient rendering ¹² ; smoother UX	Younger ecosystem; need to implement compositor.
Nano-X	Tiny window system	Very low	Minimal footprint; C-based API.	Very limited GUIs; not standard on desktop.

Toolchains and Languages

Toolchain	Languages	Pros	Cons
GCC	C, C++ (g+ +)	Mature, widely-supported; excellent target support ¹³ .	Bloat (heavy libs); slower compile.
Clang/LLVM	C, C++	Faster compile; modular; better diagnostics.	Somewhat newer for OS dev; linking intricacies.
Rust + Cargo	Rust	Memory-safety; built-in packaging; modern	Smaller ecosystem; steep learning curve; FFI needed for low-level parts.
Assembly	N/A	Ultimate control for boot, context switching.	Error-prone; tedious for large code.

We will primarily use GCC (for simplicity and compatibility) and consider Rust for select modules.

Minimal VMware Hardware/Software Specification

- **VM Type:** VMware Workstation 16 (or Player), VMware Fusion, or ESXi.
- **CPU:** x86_64 (enable VT-x if possible). 1 vCPU (enable 2+ for SMP testing later).
- **RAM:** 256–512 MB (512 MB for GUI testing).
- **Disk:** 8 GB virtual disk (IDE or SCSI); formatted with our FS (e.g. one FAT32/EXT2 partition).
- **Graphics:** 4 MB video RAM, SVGA-II (default). Enable VBE or UEFI GOP for high modes.
- **Peripherals:** PS/2 keyboard & mouse. No sound needed. Optionally E1000 NIC (for networking tests).
- **BIOS:** Standard 16-bit BIOS with A20/GDT. Enable “Guest Isolation” if VMware Tools needed.
- **EFI Option:** For Secure Boot testing, create an EFI VM (with an ESP). GRUB-EFI as first stage.
- **ISO Boot:** Boot from a generated ISO containing GRUB and our kernel. Example: `mkisofs -o myos.iso -b isolinux/isolinux.bin ...`.

This minimal setup ensures the OS can be tested consistently.

Sample Code Snippets

Below are illustrative snippets (not full programs) to show key components. They are **simplified** for clarity.

- **Bootloader (real mode, 16-bit ASM stub):**

```

; boot.S - first-stage boot sector (16-bit)
org 0x7c00
    xor ax, ax
    mov ds, ax
    mov es, ax
    ; Print character at DL=boot drive
    mov ah, 0x0E
    mov al, '!'      ; Just a test char
    int 0x10
    ; Load second stage (not shown)...
    ; Enter protected mode:
    cli
    mov eax, cr0
    or  eax, 1
    mov cr0, eax
    ; far jump to flush prefetch and load new CS
    jmp 0x08:kernel_start
times 510-($-$$) db 0
dw 0xAA55

```

• Kernel Main Loop (C):

```

// kernel.c
void kmain(void) {
    console_init();
    memory_init();
    scheduler_init();
    syscall_init();
    screen_puts("Kernel: Starting\n");
    init_syscalls();
    start_init_process(); // loads /sbin/init
    // Should never return
    for (;;) {
        asm volatile("hlt"); // idle until next interrupt
    }
}

```

• Simple Syscall Interface (x86_64, C/asm):

```

// syscall.S
.section .text
.global syscall_entry
syscall_entry:
    swapgs                // switch to kernel GS
    mov %rax, %rdi        // syscall number -> rdi
    mov %rcx, %rsi        // arg1 in rsi
    mov %rdx, %rdx        // arg2 stays
    mov %r8, %rcx         // arg3
    mov %r9, %r8          // arg4

```



```

    mov %r10, %r9          // arg5
    call do_syscall        // C function to handle syscall
    swapgs
    syscall_ret:
    sysretq                // return to user mode

// C side
uint64_t do_syscall(uint64_t num, uint64_t a1, uint64_t a2, uint64_t a3,
uint64_t a4, uint64_t a5) {
    switch (num) {
    case 1: return sys_exit(a1);
    case 2: return sys_write((int)a1, (const void*)a2, (size_t)a3);
    // ... more syscalls ...
    default: return -1;
    }
}

```

• Basic Driver Skeleton (C):

```

// drivers/keyboard.c
#include <kernel.h>
#define KBD_DATA_PORT 0x60
void keyboard_handler(void) {
    uint8_t scancode = inb(KBD_DATA_PORT);
    char c = translate_scancode(scancode);
    if (c) kbuf_putc(c); // append to keyboard buffer
    pic_eoi(IRQ_KEYBOARD);
}
void kbd_init() {
    irq_install_handler(IRQ_KEYBOARD, keyboard_handler);
    // enable keyboard device if needed...
}

```

• Minimal Compositor Loop (C/Pseudocode):

```

// gui/compositor.c
void compositor_main() {
    fb_init(); // set VBE mode, map frame buffer
    while (1) {
        for (Window *win = window_list; win; win = win->next) {
            if (win->dirty) {
                draw_window_to_fb(win);
                win->dirty = false;
            }
        }
        wait_for_vsync(); // or sleep, then reset dirty flags
        handle_input_events(); // route to windows
    }
}

```

(Actual implementations would involve more detail and structures.)

All code will be accompanied by explanations in the documentation, ensuring clarity on how each part works.

Conclusion

Creating a new OS from scratch is a *massive* undertaking, but by decomposing the problem into stages and following established principles (Unix by Ritchie ²⁷, microkernel design by Tanenbaum ¹, etc.), we can build a minimalist, modern system. Our plan balances rigor (citing seminal sources), practicality (using existing knowledge from OSDev and modern stacks), and clarity (with step-by-step instructions and diagrams). The tables and roadmap organize design decisions and milestones. By following this workflow, a functioning GUI-capable OS with low resource usage can be realized and tested in VMware. All sources and tools will be duly documented and cited, ensuring a trustworthy foundation for this original OS project.

¹ ¹⁵ ¹⁶ ¹⁷ ²³ Appendix A - The Tanenbaum-Torvalds Debate

<https://www.oreilly.com/openbook/opensources/book/appa.html>

² ⁴ ¹⁴ Microkernel vs. Monolithic Kernel - GeeksforGeeks

<https://www.geeksforgeeks.org/operating-systems/difference-between-microkernel-and-monolithic-kernel/>

³ ¹⁸ Hybrid kernel - Wikipedia

https://en.wikipedia.org/wiki/Hybrid_kernel

⁵ ⁶ ¹⁹ ²⁰ ²¹ ²² Bootloader - OSDev Wiki

<https://wiki.osdev.org/Bootloader>

⁷ Memory management - OSDev Wiki

https://wiki.osdev.org/Memory_management

⁸ Scheduling Algorithms - OSDev Wiki

https://wiki.osdev.org/Scheduling_Algorithms

⁹ System Call - GeeksforGeeks

<https://www.geeksforgeeks.org/operating-systems/introduction-of-system-call/>

¹⁰ PS/2 Keyboard - OSDev Wiki

https://wiki.osdev.org/PS/2_Keyboard

¹¹ ²⁴ 04-A.4 Journaling File Systems - Engineering LibreTexts

[https://eng.libretexts.org/Bookshelves/Computer_Science/Operating_Systems/Linux_-_The_Penguin_Marches_On_\(McClanahan\)/04%3A_Managing_Linux_Storage/4.04_Journaling_File_Systems](https://eng.libretexts.org/Bookshelves/Computer_Science/Operating_Systems/Linux_-_The_Penguin_Marches_On_(McClanahan)/04%3A_Managing_Linux_Storage/4.04_Journaling_File_Systems)

¹² Wayland

<https://wayland.freedesktop.org/>

¹³ GCC Cross-Compiler - OSDev Wiki

https://wiki.osdev.org/GCC_Cross-Compiler

²⁵ Vx32: Lightweight, User-level Sandboxing on the x86 - PDOS-MIT

<https://pdos.csail.mit.edu/papers/vx32:usenix08/>

²⁶ I fear that a lot of new Linux tools are losing the "Linux way" : r/linux

https://www.reddit.com/r/linux/comments/1ozslwz/i_fear_that_a_lot_of_new_linux_tools_are_losing/

27 [PDF] The UNIX Time-sharing System A Retrospective* - Nokia
<https://www.nokia.com/bell-labs/about/dennis-m-ritchie/retro.pdf>